AutoSense: app.js

Logic: rule-based diagnosis engine, history saving and service reminders

```
// Rule-based demo engine. Swap diagnose() for a call to your backend/ML model later.
const RULES = [
  {k:["grind","brake","squeal","squeak"], cause:"Worn brake pads or warped rotors",
    parts:["Brake pads","Brake rotors"], pri:"high", cost:[1500,6000], base:78},
  {k:["pull","steering","vibrat","shake","wobble"], cause:"Wheel misalignment or unbalanced
    tyres", parts:["Tyres","Alignment","Suspension"], pri:"medium", cost:[600,3000],
    base:70},
  {k:["start","crank","click","battery","dead"], cause:"Weak battery or faulty starter",
    parts:["Battery","Starter motor","Alternator"], pri:"medium", cost:[3000,9000],
    base:72},
  {k:["overheat","temperature","steam","coolant"], cause:"Coolant leak or failing
    thermostat/fan", parts:["Radiator","Thermostat","Water pump"], pri:"high",
    cost:[2000,12000], base:80},
  {k:["smoke","burning","oil"], cause:"Oil leak or burning oil", parts:["Gasket","Piston
    rings","Oil filter"], pri:"high", cost:[2500,25000], base:65},
  {k:["knock","ping","misfire","jerk","stall"], cause:"Ignition or fuel-delivery fault",
    parts:["Spark plugs","Injectors","Ignition coil"], pri:"medium", cost:[1000,8000],
    base:68},
  {k:["hum","whine","bearing","roar"], cause:"Worn wheel bearing", parts:["Wheel
    bearing","Hub"], pri:"medium", cost:[2500,7000], base:66},
  {k:["ac","cooling","air condition","smell"], cause:"Low AC gas or dirty cabin filter",
    parts:["AC compressor","Cabin filter","Refrigerant"], pri:"low", cost:[800,4500],
    base:62}
];
const $ = id => document.getElementById(id);
const KEY = "autosense_history", CFG = "autosense_cfg";
let current = null;

function diagnose(text, hasImg, hasAud) {
  const t = text.toLowerCase();
  const scored = RULES.map(r => ({r, hits: r.k.filter(w => t.includes(w)).length})).filter(x
     => x.hits);
  if (!scored.length) return {cause:"Not enough information", parts:["General inspection"],
    pri:"low", cost:[500,1500], conf:35, base:true};
  scored.sort((a,b) => b.hits - a.hits);
  const r = scored[0].r;
  const conf = Math.min(95, r.base + (scored[0].hits-1)*6 + (hasImg?5:0) + (hasAud?5:0));
  return {...r, conf};
}
const esc = s => s.replace(/[&<>"']/g, c =>
  ({"&":"&amp;","<":"&lt;",">":"&gt;",'"':"&quot;","'":"&#39;"}[c]));
const inr = n => "Rs." + n.toLocaleString("en-IN");
const load = () => JSON.parse(localStorage.getItem(KEY) || "[]");

$("go").onclick = () => {
  const text = $("symptom").value.trim();
  if (!text) { $("symptom").focus(); return; }
  const d = diagnose(text, $("img").files.length, $("aud").files.length);
  const model = $("model").value.trim();
  current = {date:new Date().toLocaleDateString(), model, text, cause:d.cause, pri:d.pri,
    conf:d.conf};
  const adv = d.pri==="high" ? "Avoid driving until inspected." : d.pri==="medium" ? "Book
    an inspection this week." : "Safe to drive; check at next service.";
  $("out").innerHTML = `<div class="card ${d.pri}"><h3>${esc(d.cause)}</h3>
    <p>Confidence ${d.conf}% - Priority <b>${d.pri}</b> - Estimated cost
      ${inr(d.cost[0])}-${inr(d.cost[1])}</p>
    <p>${adv}</p>${d.parts.map(p=>`<span class="tag">${esc(p)}</span>`).join("")}</div>`;
  $("find").href = "https://www.google.com/maps/search/" + encodeURIComponent("car mechanic
    near me " + (d.base?"":d.parts[0]));
  $("result").hidden = false;
  $("result").scrollIntoView({behavior:"smooth"});
};
$("save").onclick = () => { if (!current) return; const h = load(); h.unshift(current);
  localStorage.setItem(KEY, JSON.stringify(h)); render(); };
$("clear").onclick = () => { localStorage.removeItem(KEY); render(); };
["last","int"].forEach(id => $(id).onchange = () => { localStorage.setItem(CFG,
  JSON.stringify({last:$("last").value, int:$("int").value})); render(); });
$("img").onchange = e => { const f = e.target.files[0]; $("preview").innerHTML = f ? `<img
  alt="Uploaded vehicle photo" src="${URL.createObjectURL(f)}">` : ""; };

function render() {
  const c = JSON.parse(localStorage.getItem(CFG) || "{}");
  if (c.last) $("last").value = c.last; if (c.int) $("int").value = c.int;
  if (c.last) {
```

```
    const due = new Date(c.last); due.setMonth(due.getMonth() + (+c.int || 6));
    const days = Math.round((due - new Date()) / 864e5);
    $("due").textContent = days < 0 ? `Service overdue by ${-days} days.` : `Next service
      due ${due.toLocaleDateString()} (${days} days).`;
  }
  const h = load();
  $("hist").innerHTML = h.length ? h.map(x => `<li><b>${esc(x.date)}</b>
    ${esc(x.model||"Vehicle")}: ${esc(x.cause)} (${x.pri})</li>`).join("") : "<li>No saved
    repairs yet. Save a diagnosis to start your history.</li>";
}
render();
```